

SoundSoar: Predicting Music Trends with Machine Learning and Spotify

Victoria Grasso
Full Sail University
Tampa, Florida
vngrasso@student.fullsail.edu

Abstract— This study explores the application of machine learning models to predict the popularity trends of songs on Spotify, categorizing trends as "up," "down," or "stable." By leveraging the Spotify API, I collect both audio features—such as tempo and energy—and historical popularity data. The research aims to evaluate the accuracy of seven distinct models, including Random Forest, Histogram-based Gradient Boosting, and Logistic Regression, while identifying the most influential musical features that drive trend predictions. Through a structured approach that encompasses data collection, feature engineering, and comprehensive performance evaluation, the findings provide valuable insights for marketers and content creators. These insights enable more informed decision-making in an ever-changing music landscape, enhancing the ability to identify and promote songs that resonate with audiences.

Keywords—machine learning, data analysis, music popularity, Spotify Web API, feature engineering, classification

I. INTRODUCTION

In the constantly evolving landscape of digital music consumption, understanding what drives a song's success has become a focus of many record labels, artists, and marketers alike. This research seeks to assess the potential of machine learning models to forecast outcomes in classifying the popularity trends of songs on Spotify. Specifically, the study will test the hypothesis that Engineered audio features, calculated features, and classifier models can effectively predict whether a song's popularity trend on Spotify is up, down, or stable. This will be tested by assessing the performance of different classifiers in predicting these trends based on the selected features.

The primary research objectives of this project is to build a system that identifies and predicts music trends using a combination of feature engineering and machine learning models. This system is designed to assist marketers and content creators in choosing songs that are likely to trend, enhancing their ability to create impactful content ahead of the curve.

Specific research objectives include the following:

- To collect and process playlist data from a reputable source such as Spotify using the Spotify API
- To perform feature engineering by extracting relevant musical attributes like energy, tempo, and popularity.

- To develop and fine-tune classification machine learning models, including RandomForest, HistGradientBoosting, and LogisticRegression, to accurately predict the directional trend of a track (up, down, or stable).

With these research objectives in mind, the questions this study addresses are as follows: First, it examines whether machine learning models can effectively predict the trajectory of a song's popularity, determining whether it will trend upwards, downwards, or remain stable. Additionally, it investigates which musical features, such as tempo and energy, are most influential in predicting these trends. Finally, the study explores how predictions from these models can be used to help content creators select music that resonates more effectively with audiences.

II. BACKGROUND AND CURRENT SOLUTIONS

To better understand the competitive landscape and identify potential areas for improvement in SoundSoar, I conducted a detailed analysis of three key competitors: Soundcharts, Chartmetric, and Songstat. By examining user feedback from various sources, I was able to pinpoint both strengths and weaknesses of these applications. The insights gathered from this analysis will inform the development of SoundSoar, ensuring it addresses common pain points and leverages the strengths identified in the market.

Competitor	Sentiment	Feedback
Soundcharts	Positive	Soundcharts provides an extensive amount of information about streams on social media site and especially about airplay, which can be incredibly valuable for tracking the performance of artists. [11]
	Negative	For smaller artists, the platform offers limited data, making it less useful for those who don't have a large presence. Additionally, the service is pricey, with the most affordable tier starting at \$65 per month, which might be prohibitive for independent musicians. [11]
Chartmetric	Positive	"I use chartmetrics to analyze the data of the artist I manage as well as artist I consult with. I believe

		it's one of the best tools on the market for not only pulling analytics, but being a VISUAL presentation so artist can see where they may stand. Been using for Two years strong—nothing else compares!” [12]
	Negative	“Sometimes, depending on the size of the Artist and the territory he belongs to, there is a big amount of information missing. This could work in detriment to smaller independent companies interested in working with Chartmetric.” [13]
Songstats	Positive	“Songstats is one of the best music related apps out there. The information is always updated. They also provide graphics that you can share on your social media. It's inexpensive!! They are always finding ways to support artists. Also, they have added Radiostats, which has been amazing. You can see where your songs have aired. It is the entire catalog. Absolutely love this app.” [14]
	Negative	“Sorry for the low review. I really want to like this app. It could be cool to get some total metrics. And I am sure it will for some people. I wish there was more to interact with more then just a small map of the places you have been played/streamed.” [14]

Table 1 – Competitors and Market Research

Competitor analysis reveals several positive and negative trends across popular music analytics tools. On the positive side, platforms like Soundcharts and Songstats are valued for their integration with social media, which allows users to gain insights into reach and engagement. Visual appeal and clarity are also highlighted as strengths, particularly with tools like Chartmetric, as their presentation makes complex data accessible and easy to interpret, helping users make informed decisions. Additionally, the comprehensive and up-to-date information provided by platforms such as Soundcharts and Songstats, especially on airplay and other metrics, is appreciated for its role in helping musicians expand their reach. On the other hand, several negative trends were noted. Soundcharts and Chartmetric are often criticized for not catering to smaller or independent musicians, offering limited data or features that may not justify the cost. The high expense of these platforms is another common complaint, with smaller artists or independent labels finding pricing tiers like Soundcharts' \$65 per month plan to be prohibitively expensive. Furthermore, users have pointed out gaps in data, particularly with Chartmetric, as it sometimes lacks comprehensive information for smaller artists or certain territories, which limits its overall usefulness.

III. LITERATURE REVIEW

This literature review examines key research that informs the current study on predicting music popularity trends through engineered audio features and classifier models. The rapidly changing landscape of digital music distribution, particularly through platforms like Spotify, necessitates a further understanding of what factors could influence a song's popularity. To build on established findings, this review focuses on several core studies. Kang, Mandulak, and Szymanski (2022) [7] analyze professional musicians' success, identifying predictors that set the stage for further exploration of predictive features in popular music. Araujo, Cristo, and Giusti (2019) [8] contribute insights into the predictive power of music charts, shedding light on the underlying patterns in listener behavior across chart-topping songs. Sciandra and Spera (2022) [9] approach Spotify data with a Beta GLMM model, introducing statistical rigor to trend analysis. Finally, Saragih (2023) [10] examines the correlation between Spotify's audio features and song popularity in the Indonesian market, revealing valuable insights into audio-driven popularity metrics. Together, these studies provide a methodological and conceptual foundation that guides the development of an effective predictive algorithm in this research.

The first literature examines the use of machine learning models, notably XGBoost, to predict professional musician success based on a blend of features such as popularity metrics, genre classification, and collaboration networks [7]. Kang, Mandulak, and Szymanski's study demonstrates the potential of complex tree-based algorithms in identifying genre-based patterns and artist trajectories. XGBoost, known for its gradient boosting framework, excels in handling large feature sets and nuanced interactions, making it highly effective in predicting long-term artist success across various genres. By focusing on features like collaboration data and genre affiliation, the study reveals that musicians' genre choices and collaborative networks can significantly impact their popularity, highlighting the importance of these factors in predictive modeling.

Inspired by this approach, my work diverges to focus on track-level data, specifically aiming to predict a song's trend—whether it rises, falls, or remains stable over time. I employ seven models, beginning with Random Forest and HistGradientBoosting for their strengths in managing complex feature interactions and capturing feature importance in musical data. These ensemble methods allow for a broader exploration of individual track attributes, enhancing the granularity of trend prediction. The inclusion of models such as SVM and Logistic Regression further allows for a nuanced approach, with each model contributing specific advantages. For example, KNN supports proximity-based classification, while LDA aids in feature selection through dimensionality reduction.

While this multi-model approach provides flexibility and a robust framework for song trend prediction, it has its disadvantages. The diversity of models requires extensive computational resources and time due to the complex process of tuning and optimizing each model, particularly with Grid Search for hyperparameter tuning. Additionally, while effective for in-depth analysis, this approach could impact real-time prediction

efficiency, as model evaluation and selection demand significant processing power.

The second study introduces a methodology for predicting a song's likelihood of appearing in Spotify's Top 50 Global ranking, utilizing Support Vector Machines (SVM) with RBF kernels to capture complex, non-linear relationships in audio and engagement metrics [8]. By accurately predicting track popularity, this research highlights significant marketing advantages for artists, labels, and streaming platforms, enabling them to strategically plan release schedules and promotions. Beyond marketing, such predictions can benefit playlist curators and advertisers by helping them understand listener trends, ultimately enhancing user engagement.

Similarly, my project seeks to predict track popularity, but it expands beyond the Top 50 to include a variety of influential playlists, such as Spotify's Today's Top Hits and Big on the Internet, as well as Viral Hits, TikTok Monthly, and others. By training on this broader dataset, my approach captures popularity trends across diverse, social media-influenced contexts. Among the seven models I employ, SVM plays a key role for its effective handling of high-dimensional data, complementing ensemble methods like Random Forest and HistGradientBoosting. Together, these models enable detailed popularity predictions, offering content creators timely insights to strategically plan their releases and maximize audience engagement based on anticipated trends.

In essence, while both studies focus on predicting popularity, my work specifically aims to equip content creators with valuable marketing data to help them leverage timing and trends for their own benefit, rather than focusing on data tailored for artists and labels. This shift underscores the practical applications of popularity predictions in fostering successful content creation strategies within the ever-evolving landscape of digital music distribution.

The third literature examines the integration of Spotify's Web API with R to analyze music data, particularly focusing on song popularity as determined by recent plays and total streams [9]. The authors employ a Beta Generalized Linear Mixed Model (GLMM) to investigate the relationship between the different audio features—such as danceability and energy—and track popularity, emphasizing the importance of understanding these determinants in a data-driven music landscape. Their study highlights key audio characteristics that influence popularity, providing valuable insights for those interested in predicting a song's success.

In my research, I opted not to utilize the GLMM framework, choosing instead to implement a seven-model approach that leverages diverse algorithms to enhance predictive accuracy and capture complex relationships within the data. This methodology allows for a comprehensive analysis of the audio features that are also discussed in the GLMM study, such as energy and loudness, which are relevant to my investigation of track popularity. By employing this range of models, I aim to provide a robust understanding of how different factors can contribute to a song's performance, ultimately equipping content creators with actionable insights to navigate the evolving digital music landscape.

The final literature contributes to the growing field of machine learning in music analytics by employing both classification and regression techniques to predict song popularity based on Spotify's audio features [10]. The study's use of ensemble methods like the Random Forest Classifier and Extra Trees Regressor demonstrates the value of these techniques in analyzing large datasets, with cross-validation and hyperparameter tuning enhancing model robustness. Focused on the Indonesian market, this research highlights the unique musical preferences and cultural factors that can influence song popularity in Southeast Asia. Indonesia's rapidly expanding music industry, projected to reach \$343 million by 2025, underscores the importance of understanding local audio feature trends to inform both artists and marketers about consumer behavior.

This context is particularly significant as prior research indicates that different countries may exhibit varying determinants of musical success. However, the study omits factors such as genre or artist familiarity, suggesting avenues for future research. In my work, I expand on these methodologies by applying similar machine learning models but with a focus on predictive accuracy and feature selection to explore song popularity in diverse playlist contexts.

IV. METHODOLOGY

This study employs a quantitative methodology to develop an effective predictive model for forecasting the popularity of tracks on curated Spotify playlists. By leveraging data-driven insights combined with machine learning techniques, this approach enhances the accuracy of popularity predictions. This approach was selected due to its ability to combine data-driven insights with machine learning techniques, which can significantly enhance the accuracy of popularity predictions. The methodology consists of several key steps: data collection, feature engineering, model selection, and performance evaluation.

In the data collection phase, I utilized Spotify's Web API to gather detailed audio features and popularity metrics for tracks within specified playlists. This included not only attributes such as tempo, energy, valence, danceability, and speechiness but also the current popularity scores assigned by Spotify, which range from 0 to 100. To enhance the dataset, I incorporated historical popularity data, creating a time series that allows for comprehensive trend analysis.

Feature engineering was a crucial step in improving the predictive power of the model. I extracted raw audio features from the Spotify API and engineered additional metrics based on historical popularity. This dual approach enables the model to account for both the immediate musical attributes and the dynamic nature of popularity trends over time.

For model selection, I experimented with multiple machine learning algorithms, including ensemble-based classifiers like Random Forest and Histogram-based Gradient Boosting, as well as other techniques such as Support Vector Machines and K-Nearest Neighbors. This diverse selection of models allows for a well-rounded evaluation of performance and helps identify the most effective algorithm for predicting trends.

Finally, the performance evaluation phase involved testing the models using key metrics such as accuracy, precision, recall, and F1-score. This comprehensive evaluation process ensures that the model can generalize well to unseen data and accurately predict whether a track will trend upwards, downwards, or remain stable.

By employing this structured methodology, the study aims to provide valuable insights into the factors that influence music popularity and assist content creators in selecting tracks that resonate with audiences. The combination of detailed data collection, feature engineering, and thorough evaluation enables a robust analysis that directly supports the project’s goals.

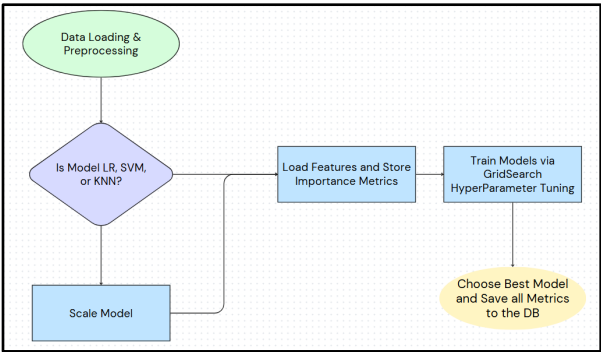


Figure 1 – Methodology Flow Diagram

Data was sourced using Spotify’s Web API [15], which provided detailed audio features and popularity metrics for tracks within specified playlists. Features extracted included tempo, energy, valence, danceability, and speechiness, among others, along with the current popularity score assigned by Spotify, which ranges from 0 to 100. The dataset was then expanded by including historical popularity data for each track, creating a time series of popularity scores to enable trend analysis.

To facilitate this data collection, I developed a web application using Django, which interfaced with the Spotify API to retrieve audio features and popularity metrics. The collected data was stored in a MySQL database, providing a structured and efficient way to manage and query the information. To automate the data retrieval process, I implemented a PowerShell script that runs on a scheduled basis through Windows Task Scheduler. This script makes API calls to Spotify, retrieving the latest audio features and updating the database with new and historical popularity data. Additionally, whenever a new model is created, the data from the MySQL database is saved as a CSV file. This practice ensures the reproducibility of results and provides a consistent reference point for future analyses, supporting comprehensive trend evaluation and accurate predictions in SoundSoar.

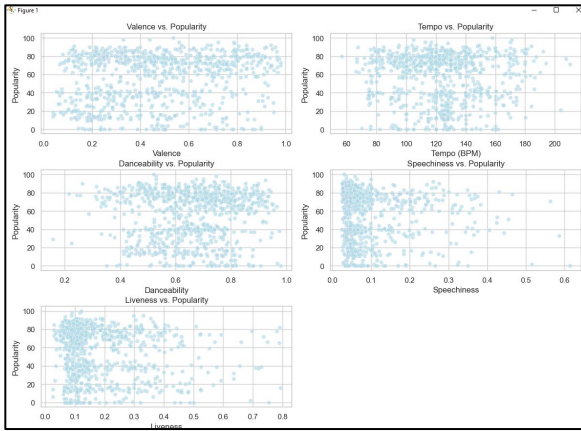
Feature engineering played a vital role in enhancing the predictive power of my model. I began by extracting raw audio features directly from Spotify’s Web API [15], including attributes such as tempo, speechiness, danceability, valence, and liveness. These features were then combined with calculated track features in a preliminary analysis, which will be discussed in the next section. I selected the previously mentioned audio

features based on their high importance scores from my feature importance analysis.

Valence	Sourced from Spotify Web API: “A measure from 0.0 to 1.0 describing the musical positiveness conveyed by a track. Tracks with high valence sound more positive (e.g. happy, cheerful, euphoric), while tracks with low valence sound more negative (e.g. sad, depressed, angry).” [15]
Tempo	Sourced from Spotify Web API: “The overall estimated tempo of a track in beats per minute (BPM). In musical terminology, tempo is the speed or pace of a given piece and derives directly from the average beat duration.” [15]
Speechiness	Sourced from Spotify Web API: “Speechiness detects the presence of spoken words in a track. The more exclusively speech-like the recording (e.g. talk show, audio book, poetry), the closer to 1.0 the attribute value. Values above 0.66 describe tracks that are probably made entirely of spoken words. Values between 0.33 and 0.66 describe tracks that may contain both music and speech, either in sections or layered, including such cases as rap music. Values below 0.33 most likely represent music and other non-speech-like tracks.” [15]
Danceability	Sourced from Spotify Web API: “Danceability describes how suitable a track is for dancing based on a combination of musical elements including tempo, rhythm stability, beat strength, and overall regularity. A value of 0.0 is least danceable and 1.0 is most danceable.” [15]
Liveness	Sourced from Spotify Web API: “Detects the presence of an audience in the recording. Higher liveness values represent an increased probability that the track was performed live. A value above 0.8 provides strong likelihood that the track is live.”[15]

Table 2 – Spotify Web API Audio Features

However, upon conducting Cartesian correlations to directly compare each feature with track popularity, I found limited correlation. This suggests that while these features may not predict outright popularity, they could influence the duration of a track's trend. Thus, the absence of correlation could be advantageous, contributing to a balanced perspective on how these features interact with the dynamics of track popularity.



In addition to the raw audio attributes, I engineered features based on the historical popularity scores provided by Spotify, which are updated frequently. These history-based features aimed to capture the dynamic trends in a track's popularity:

Velocity	<pre> historical_popularity = self.get_historical_popularity() if len(historical_popularity) < 2: self.velocity = 0 return recent = np.array(historical_popularity[-2:]) rate_of_change = (recent[1] - recent[0]) / recent[0] if recent[0] != 0 else 0 self.velocity = rate_of_change </pre>
Current Popularity	This value represents the latest popularity metric obtained from the Spotify API or the most recent update.
Median Popularity	<pre> historical_popularity = self.get_historical_popularity() self.median_popularity = np.median(historical_popularity) if historical_popularity else None </pre>
Mean Popularity	<pre> historical_popularity = self.get_historical_popularity() self.mean_popularity = np.mean(historical_popularity) if historical_popularity else None </pre>
STD Popularity	<pre> historical_popularity = self.get_historical_popularity() self.std_popularity = np.std(historical_popularity) if historical_popularity else Non </pre>
Retrieval Frequency	<pre> if self.velocity > 0.1: self.retrieval_frequency = 'high' elif 0.01 < self.velocity <= 0.1: self.retrieval_frequency = 'medium' else: self.retrieval_frequency = 'low' </pre>

Trend (target)	<pre> historical_popularity = self.get_historical_popularity() if len(historical_popularity) < 3: self.trend = 'stable' return mean_popularity = self.mean_popularity if self.current_popularity > mean_popularity: if self.velocity > 0: self.trend = 'up' # Upward trend elif self.velocity == 0: self.trend = 'stable' # No significant change else: self.trend = 'down' # Current popularity is decreasing elif self.current_popularity < mean_popularity: if self.velocity < 0: self.trend = 'down' # Downward trend elif self.velocity == 0: self.trend = 'stable' # No significant change else: self.trend = 'up' # Current popularity is improving else: self.trend = 'stable' # Current popularity equals mean </pre>
----------------	---

Table 3 – Popularity History Based Features

By incorporating both raw audio features and engineered popularity-based metrics, the feature set provides a comprehensive and dynamic representation of a track's current standing and potential trajectory. These combined features are designed not only to capture the immediate musical attributes, such as tempo and danceability, but also to reflect the evolving trends in popularity through time-series analysis. The velocity and trend metrics, derived from historical popularity data, play a critical role in predicting shifts in a track's popularity—whether upward, downward, or stable. This dual focus on static audio features and dynamic trend-based indicators allows the model to make more informed predictions about future track performance. Additionally, the retrieval frequency feature ensures that tracks with fast-rising popularity receive timely updates, enabling real-time responsiveness. Overall, the integration of these engineered features enhances the model's ability to predict both the current and future popularity trends of tracks on Spotify. The sample data below illustrates how these features and the target variable are structured for the machine learning algorithms:

track_valence	0.104
track_tempo	141.981
track_speechiness	0.0561
track_danceability	0.47
track_liveness	0.175

velocity	0.0001
current_popularity	83
median_popularity	83
mean_popularity	82.703
std_popularity	0.4566
retrieval_frequency	low
trend	stable

Table 4 – Sample Data

To predict track popularity trends, I experimented with several machine learning models, each bringing unique strengths to the task. The model selection process involved starting with two ensemble-based classifiers—Random Forest Classifier and Histogram-based Gradient Boosting Classifier (HistGradientBoosting)—and progressively incorporating additional models to improve performance and ensure comprehensive evaluation. These models were selected based on their ability to handle missing values, scalability to large datasets, and capacity for fine-tuning. Over time, five more models were introduced, enabling a well-rounded comparison of performance across different algorithmic approaches.

Random Forest Classifier: A robust ensemble method that builds multiple decision trees and averages their predictions to improve accuracy and reduce overfitting. Its ability to handle high-dimensional data and natively manage missing values during training made it a logical choice for my dataset. Random Forest also offers built-in feature importance ranking, aiding in the interpretability of results.

Histogram-based Gradient Boosting Classifier (HistGradientBoosting): A variation of gradient boosting that utilizes histogram binning to speed up training, making it particularly efficient for large datasets. It is also capable of handling missing values natively, offering an ideal solution for my dataset with potential gaps in features. The gradient boosting approach helps refine predictions iteratively, minimizing prediction errors in a sequential manner.

After evaluating the initial performance of these two models, I expanded the range of algorithms to experiment with different classification techniques:

Support Vector Machine (SVM): SVM was added for its ability to find optimal hyperplanes that maximize the margin between classes. While SVM does not handle missing values natively, data imputation and scaling were applied prior to its use, ensuring consistent performance across feature sets. Its strength lies in its ability to handle complex relationships in the data, especially when kernel transformations are applied.

Linear Discriminant Analysis (LDA): LDA was selected as a linear classification model that assumes normally distributed

data, making it an interesting contrast to the tree-based models. LDA excels in situations where there are clear class separations, and while it requires complete data, preprocessing steps ensure its applicability.

Extra Trees Classifier: This model works similarly to Random Forest but differs in how it selects splits within the trees. By randomly selecting splits rather than choosing the best split, Extra Trees introduces more variance and potentially greater robustness, particularly in noisy datasets like mine. Its ability to handle missing data and large feature spaces made it an excellent addition.

K-Nearest Neighbors (KNN): KNN was introduced to evaluate the performance of a non-parametric model that makes predictions based on the majority class of its nearest neighbors. Although KNN requires preprocessing, including data scaling and imputation, it offers an intuitive approach to classification and can perform well in low-dimensional spaces.

Logistic Regression: Finally, Logistic Regression was used as a baseline model to compare the performance of more complex models. Despite being a simple linear classifier, it provides valuable insights when combined with regularization techniques (like L2 regularization) and serves as a strong benchmark for overall accuracy.

To maximize model performance, I implemented hyperparameter tuning using Grid Search with cross-validation. This approach optimizes the models by systematically exploring a range of hyperparameters to enhance their predictive capabilities. Given that I experimented with various parameters throughout the project, I selected several key hyperparameters for the final models, with the possibility of further tuning as the research progresses.

For the Random Forest Classifier, the hyperparameters included the number of estimators (`n_estimators`) set at [100, 200], maximum tree depth (`max_depth`) at [None, 20], minimum samples required to split (`min_samples_split`) at [2, 5], minimum samples per leaf (`min_samples_leaf`) at [1, 2], maximum features (`max_features`) set to ['sqrt'], bootstrap set to [True], and criterion as ['gini', 'entropy'].

The Histogram-based Gradient Boosting Classifier (HistGradientBoosting) hyperparameters were defined as follows: number of boosting iterations (`max_iter`) at [100, 200], learning rate (`learning_rate`) at [0.01, 0.1], maximum tree depth (`max_depth`) at [None, 5], minimum samples per leaf (`min_samples_leaf`) at [1], maximum bins (`max_bins`) at [255], L2 regularization (`l2_regularization`) at [0, 0.1], and early stopping set to [True].

For Logistic Regression, the selected hyperparameters included the inverse regularization strength (`C`) at [500, 700, 900], penalty set to ['l2'], solver options as ['saga', 'liblinear', 'lbfgs'], intercept scaling (`intercept_scaling`) at [1, 5], and fit intercept options as [True, False].

In the case of the Support Vector Machine (SVM), the hyperparameters comprised the regularization parameter (`C`) set at [3, 5, 10], kernel coefficient (`gamma`) options as ['scale', 'auto'], and kernel types as ['rbf', 'linear', 'poly'].

For Linear Discriminant Analysis (LDA), the hyperparameters included solver options as ['svd', 'lsqr', 'eigen'] and priors set to [None, [0.5, 0.5], [0.3, 0.7]].

The Extra Trees Classifier featured hyperparameters such as the number of estimators (`n_estimators`) set at [100, 200], minimum samples per leaf (`min_samples_leaf`) at [1, 2], maximum features (`max_features`) set to ['sqrt'], bootstrap options at [True, False], and criterion as ['gini', 'entropy'].

Finally, for K-Nearest Neighbors (KNN), the hyperparameters were defined as the number of neighbors (`n_neighbors`) set at [11, 15, 21], weights set to ['distance'], algorithm options as ['auto'], leaf size at [5, 10, 20], and metric choices as ['euclidean', 'manhattan'].

Using GridSearchCV with a 5-fold cross-validation, each model was trained and evaluated, providing insight into the best combination of hyperparameters for each model. The grid search utilized accuracy as the primary evaluation metric, selecting hyperparameters that maximized predictive performance on the validation sets.

The model training process also included essential data preprocessing steps, such as imputation for handling missing values and feature scaling, particularly for models sensitive to feature magnitudes, like Logistic Regression, SVM, and KNN. After identifying the optimal hyperparameters, the best models were stored for future comparisons and predictions.

Once hyperparameter tuning was finalized, I evaluated the best-performing models using a holdout test set that accounted for 20% of the original dataset. This evaluation was crucial for understanding how well the models could generalize to unseen data. To facilitate this comparison, I focused on several key metrics. Accuracy was one of the primary measures, representing the proportion of correctly predicted trends—whether increasing, decreasing, or stable—relative to the total predictions made. This straightforward metric offers a clear indication of overall model effectiveness. Additionally, I generated a classification report that provided a detailed analysis of model performance, including precision, recall, and F1-score for each trend category.

Precision indicates how accurately the model made positive predictions, calculated by dividing the number of true positive predictions by the total number of positive predictions (true positives plus false positives). Recall, on the other hand, measures the model's capability to identify all relevant instances within the dataset, determined by the ratio of true positive predictions to the total actual positives (true positives plus false negatives). The F1-score, which combines both precision and recall into a single metric, serves as the harmonic mean of the two, thus balancing their concerns, especially in situations involving imbalanced classes. Furthermore, I visualized a confusion matrix to illustrate the counts of correct and incorrect predictions across each trend category, providing deeper insights into the model's strengths and weaknesses and revealing which classes were often confused with one another.

After identifying the most effective model, I utilized the `joblib` library to save it alongside the feature names and the imputer used during preprocessing. This preservation ensures that the model can be easily deployed for future predictions

without the need to retrain from scratch. The saved model file serves as an asset for ongoing research, enabling quick access to predictions and further experimentation. To maintain the model's accuracy and adaptability to new trends in music, I plan to retrain it periodically. A scheduled task will be implemented to automate the retrieval of updated data from the Spotify API, allowing the model to learn from the latest trends in track popularity. This continuous training will help ensure the model remains relevant and effective in a rapidly changing music landscape, providing creators with reliable predictions as they seek to align their content with current trends.

V. RESULTS

To ensure the model operates with up-to-date data, I have integrated a system that updates the popularity history of tracks daily. Each track is initially set to a low-frequency tracking mode, but if the track's velocity (a key feature indicating upward movement) becomes positive, the system dynamically adjusts its tracking frequency to either medium or high. This ensures that fast-rising tracks receive more frequent updates, providing more accurate trend predictions.

I also developed an automated workflow within my Django app to streamline model training, evaluation, and data storage. This system runs all seven models, evaluates their performance, and then saves critical information, including feature importance scores, model parameters, and model files. Each model is stored along with a README file, a `.pkl` file containing the model itself, and a `.csv` file filled with relevant evaluation data. The `joblib` library is used for serializing and saving trained models, ensuring reproducibility across experiments.

During the data preprocessing phase, missing values are handled using `SimpleImputer`, while `StandardScaler` is applied to scale numerical features, particularly to optimize performance for models like LogisticRegression, SVM, and KNN. GridSearchCV is utilized to fine-tune hyperparameters, optimizing aspects such as estimators, learning rates, and depth configurations for models like RandomForest and HistGradientBoosting. Feature importance is extracted for models like RandomForest and Extra Trees, while for others, including SVM and HistGradientBoosting, permutation importance is computed using the `permutation_importance` method from Scikit-learn.

Initially, I began the experimentation process with RandomForest and HistGradientBoosting, and gradually added more complex models such as SVM, LDA, ExtraTrees, and KNN as the project advanced. This iterative approach enabled continuous refinement and experimentation with different algorithms. To date, I have created and saved approximately 70 different models through various iterations and experiments. Each model's performance is evaluated using metrics such as accuracy, classification reports, and confusion matrices, providing insights into areas for improvement. The results tables below display the performance of the seven most recent models, along with the average performance metrics of all the models developed throughout the project.

Model	Accuracy	Precision	F1	Recall	Confusion Matrix	Parameters
RF	.86	.86	.85	.86	[[34, 15, 0], [3, 97, 0], [2, 3, 6]]	bootstrap: True, criterion: gini, max_depth: 20, max_features: sqrt, min_samples_leaf: 1, n_estimators: 200
LR	.69	.69	.72	.65	[[13, 36, 0], [3, 92, 5], [0, 5, 6]]	C: 500, fit_intercept: False, intercept_scaling: 1, penalty: l2, solver: liblinear
HGB	.86	.87	.85	.86	[[33, 16, 0], [1, 98, 1], [0, 5, 6]]	early_stopping: True, l2_regularization: 0, learning_rate: 0.01, max_bins: 255, max_depth: 5, max_iter: 100, min_samples_leaf: 1
SVM	.78	.78	.77	.79	[[28, 21, 0], [8, 92, 0], [1, 5, 5]]	C: 3, gamma: scale, kernel: rbf
LDA	.70	.72	.64	.70	[[10, 39, 0], [3, 96, 1], [0, 5, 6]]	priors: None, solver: lsqr
ET	.83	.84	.82	.83	[[33, 16, 0], [6, 94, 0], [2, 3, 6]]	bootstrap: True, criterion: gini, max_features: sqrt, min_samples_leaf: 1, n_estimators: 200
KNN	.81	.81	.80	.81	[[29, 20, 0], [6, 94, 0], [1, 4, 6]]	algorithm: auto, leaf_size: 5, metric: manhattan, n_neighbors: 21, weights: distance

Table 5 – Active Model Performance (Sept 19 – Oct 19, 2024)

Next, we will examine the feature importance of the active models. Identifying which musical features significantly influence predictions will help us understand the models' effectiveness and guide content creators in making informed music selections.

Feature Name	Importance Score
track_valence	0.0693
track_tempo	0.0776
track_speechiness	0.0660
track_danceability	0.0769
track_liveness	0.0619
velocity	0.1331
current_popularity	0.0979
median_popularity	0.0947
mean_popularity	0.1189
std_popularity	0.1440
retrieval_frequency	0.0597

Table 6 – RandomForest Feature Importance

Feature Name	Importance Score
track_valence	0.0081
track_tempo	0.0926
track_speechiness	0.0166
track_danceability	0.3307
track_liveness	0.0759
velocity	10.7683

current_popularity	12.4955
median_popularity	9.1245
mean_popularity	21.6259
std_popularity	1.1411
retrieval_frequency	1.1344

Table 7 – LogisticRegression Feature Importance

Feature Name	Importance Score
track_valence	0.0077
track_tempo	0.0195
track_speechiness	0.0153
track_danceability	0.0312
track_liveness	0.0050
velocity	0.1517
current_popularity	0.1469
median_popularity	0.0031
mean_popularity	0.0280
std_popularity	0.0841
retrieval_frequency	0.0000

Table 8 – HistGradientBoosting Feature Importance

Feature Name	Importance Score
track_valence	0.0073
track_tempo	0.0234
track_speechiness	0.0156
track_danceability	0.0245
track_liveness	0.0187
velocity	0.0964
current_popularity	0.0903
median_popularity	0.0592
mean_popularity	0.0691
std_popularity	0.0078
retrieval_frequency	0.0323

Table 9 – Support Vector Machine Feature Importance

Feature Name	Importance Score
track_valence	4.4302
track_tempo	0.1921
track_speechiness	10.1115
track_danceability	36.5774
track_liveness	13.4970
velocity	47.3958
current_popularity	0.2769
median_popularity	0.5377
mean_popularity	0.9033
std_popularity	0.5867
retrieval_frequency	2.9345

Table 10 – Linear Discriminant Analysis Feature Importance

Feature Name	Importance Score
track_valence	0.0779
track_tempo	0.0730
track_speechiness	0.0745
track_danceability	0.0760
track_liveness	0.0620
velocity	0.1034
current_popularity	0.0993

median popularity	0.1086
mean popularity	0.1185
std popularity	0.1075
retrieval frequency	0.0994

Table 11 – ExtraTrees Feature Importance

Feature Name	Importance Score
track_valence	0.1631
track_tempo	0.1698
track_speechiness	0.1248
track_danceability	0.1706
track_liveness	0.1403
velocity	0.0977
current popularity	0.1514
median popularity	0.1553
mean popularity	0.1564
std popularity	0.0542
retrieval frequency	0.0233

Table 12 – KNN Feature Importance

Finally, I wanted to include all models and present the average results to provide a comprehensive overview. This will allow us to reflect on these findings in the discussion section that follows.

Model	# of Models	Accuracy	Precision	F1	Recall
RF	24	0.89	0.89	0.88	0.89
LR	10	0.84	0.86	0.84	0.84
HGB	12	0.89	0.89	0.88	0.89
SVM	10	0.88	0.89	0.87	0.88
LDA	5	0.77	0.71	0.71	0.77
ET	5	0.85	0.86	0.84	0.85
KNN	5	0.77	0.80	0.73	0.77

Table 13 Average Model Performance (Sept 19 – Oct 19, 2024)

VI. DISCUSSION

The results confirm that popularity metrics play a key role in predicting song trends, with models like RandomForest and HistGradientBoosting leading in accuracy at 86%. ExtraTrees followed closely at 83% accuracy, leveraging features like “mean” and “median” popularity to effectively capture subtle trend patterns. SVM and KNN, achieving 78% and 81% accuracy, respectively, also showed merit. SVM’s structure helped it separate complex trend patterns, while KNN’s choice of 21 neighbors allowed it to accurately group nearby trends. LogisticRegression and LDA, while more interpretable, lagged behind (69% and 70% accuracy), showing that linear models struggle to capture complex “up” versus “down” shifts in trends.

Feature importance analysis highlighted which aspects of a track most influenced predictions. RandomForest emphasized “standard deviation in popularity” and “velocity,” showing that popularity consistency and song energy are closely tied to trend stability. Similarly, HistGradientBoosting leaned on “velocity” and “current popularity” for steady predictions. Models like SVM and KNN placed higher importance on rhythmic features like “tempo” and “danceability,” indicating their reliance on audio attributes to define trend categories.

Overall, models prioritizing popularity-based features—RandomForest, HistGradientBoosting, and SVM—consistently outperformed, underlining popularity’s value as a trend predictor. Earlier in the study, when popularity metrics varied less, models showed steadier accuracy. As data evolved, the highest-performing models adjusted well to shifting popularity trends, further validating the central role of popularity metrics. By contrast, models relying more on audio features, like LDA and KNN, were less adaptable to trend shifts. These findings suggest that in the context of this study, popularity fluctuations are essential for tracking and predicting trend patterns accurately.

VII. CONCLUSION AND FUTURE CONSIDERATIONS

This study set out to answer three primary questions: (1) Can machine learning models accurately classify a song's popularity trend on Spotify as "up," "down," or "stable"? (2) Which musical and historical popularity features contribute most to these trend predictions? (3) How can these insights help content creators identify tracks likely to resonate with audiences?

My findings reveal that machine learning models, particularly RandomForest and HistGradientBoosting, excel at predicting trend directions, with strong accuracy rates. Popularity-based features, such as "velocity" and "current popularity," showed the highest correlations with trend shifts, underscoring that popularity metrics play a more significant role in capturing trend dynamics than audio features alone. By focusing on historical popularity rather than direct audio features, this study developed a model-driven approach that can provide targeted trend forecasting for music. The integration of automated processes enables the model to update predictions continuously, offering marketers and content creators timely, adaptive insights that align with current music trends. This approach empowers these stakeholders to stay ahead of shifts and select content likely to resonate with audiences.

The project leaned heavily on popularity-related features over track attributes like tempo and danceability, which require less storage but are less predictive of trend changes. While expanding storage for a more comprehensive view of historical popularity could provide richer insights, data storage costs may impose limitations. Moreover, the complexity of models such as RandomForest and HistGradientBoosting presents challenges in interpretability, making it difficult to communicate results effectively to non-technical users.

Future work could focus on refining models like KNN and LDA, which prioritize track attributes. This could provide trend predictions that bypass extensive storage requirements, potentially enhancing accessibility without sacrificing predictive quality. Additionally, exploring new data sources beyond Spotify and experimenting with neural networks or ensemble methods might yield even better predictive accuracy. Improving model interpretability would also make trend forecasting more accessible for a broader range of users.

Several elements went particularly well in this project. The automated workflow for model training, evaluation, and data storage streamlined the approach, supporting continuous model updates and easy reproducibility. The choice to prioritize

popularity metrics over audio attributes proved beneficial, as these features were more strongly correlated with trend shifts, enhancing prediction accuracy. However, limitations in Spotify API access and storage capacity constrained the project's ability to maintain a comprehensive dataset, reducing data richness for certain models. Balancing model complexity and interpretability also proved challenging, as high-performing models like RandomForest were harder to interpret for non-expert users.

This project highlighted the importance of adaptive modeling and effective data management. In future projects, I would retain the automated workflow and the emphasis on popularity-based features, as both were instrumental in achieving reliable predictions. However, planning for a scalable storage solution to better support historical data collection would strengthen the research. I would also explore models that emphasize track features alongside popularity metrics for a more balanced approach to trend prediction. This dual focus could provide deeper insights without requiring extensive data storage, further advancing trend forecasting capabilities.

REFERENCES

- [1] Spotify AB, "Spotify Web API," [Online]. Available: <https://developer.spotify.com/documentation/web-api>.
- [2] Plotly, "Getting Started with Plotly in Python," [Online]. Available: <https://plotly.com/python/getting-started/>.
- [3] Pandas Development Team, "Pandas Documentation," [Online]. Available: <https://pandas.pydata.org/docs/>.
- [4] Scikit-learn Developers, "Scikit-learn Documentation," [Online]. Available: <https://scikit-learn.org/stable/>.
- [5] Django Software Foundation, "Django Documentation," [Online]. Available: <https://docs.djangoproject.com/en/stable/>.
- [6] Amazon Web Services, "AWS Documentation," [Online]. Available: <https://aws.amazon.com/documentation/>.
- [7] I. Kang, M. Mandulak, and B. K. Szymanski, "Analyzing and predicting success of professional musicians," *Scientific Reports*, vol. 12, no. 21838, 2022. doi: 10.1038/s41598-022-25430-9.
- [8] C. V. S. Araujo, M. A. P. Cristo, and R. Giusti, "Predicting music popularity using music charts," In *Proceedings of the SBCM Conference*, 2019. doi: 10.5753/sbcm.2019.10436.
- [9] M. Sciandra and I. C. Spera, "A model-based approach to Spotify data analysis: a Beta GLMM," *Journal of Applied Statistics*, vol. 49, no. 1, pp. 214–229, 2022. doi: 10.1080/02664763.2020.1803810.
- [10] H. S. Saragih, "Predicting song popularity based on Spotify's audio features: insights from the Indonesian streaming users," *Journal of Management Analytics*, vol. 10, no. 4, pp. 693–709, 2023. doi: 10.1080/23270012.2023.2239824.
- [11] M. Witthaus, "Soundcharts Review – Powerful Music Analytics for Professionals," YouTube, Apr. 10, 2021. [Online]. Available: <https://www.youtube.com/watch?v=48fwZQ7Cqms>
- [12] "Chartmetric Reviews," Trustpilot. <https://www.trustpilot.com/review/chartmetric.com> (accessed Aug. 16, 2024).
- [13] "Chartmetric Reviews & Ratings 2023," G2. <https://www.g2.com/products/chartmetric/reviews> (accessed Aug. 16, 2024).
- [14] "Songstats - Music Analytics," Apple App Store. <https://apps.apple.com/us/app/songstats-music-analytics/id1436000800?see-all=reviews> (accessed Aug. 16, 2024).
- [15] "Get audio features," Spotify Developer. <https://developer.spotify.com/documentation/web-api/reference/get-audio-features>